DTS103TC
Data Structure and Algorithms

Lab 7: Greedy Algorithms

Dr. Qi Chen, Dr. Di Zhang, Dr. Md Maruf Hasan,
Dr. Xuechen Liu, Dr. Guangyu Ren and Dr. Bin Chen
School of AI and Advanced Computing

Learning outcome

- Able to implement Greedy algorithms with Python
  - Coin Change
  - Activity Selection
  - Fractional Knapsack
  - Minimum Number of Platforms

Coin Change Problem

Suppose we have 3 types of coins

0.1

0.5

1.0

Minimum number of coins to make
0.8, 1.0, 1.4 ?

Greedy method

Coin Change

- Given a value N, if we want to make change for N cents, and we have infinite supply of each of $S = \{S_1, S_2, .., S_m\}$ valued coins. Find the fewest number of coins that you need to make up that amount.

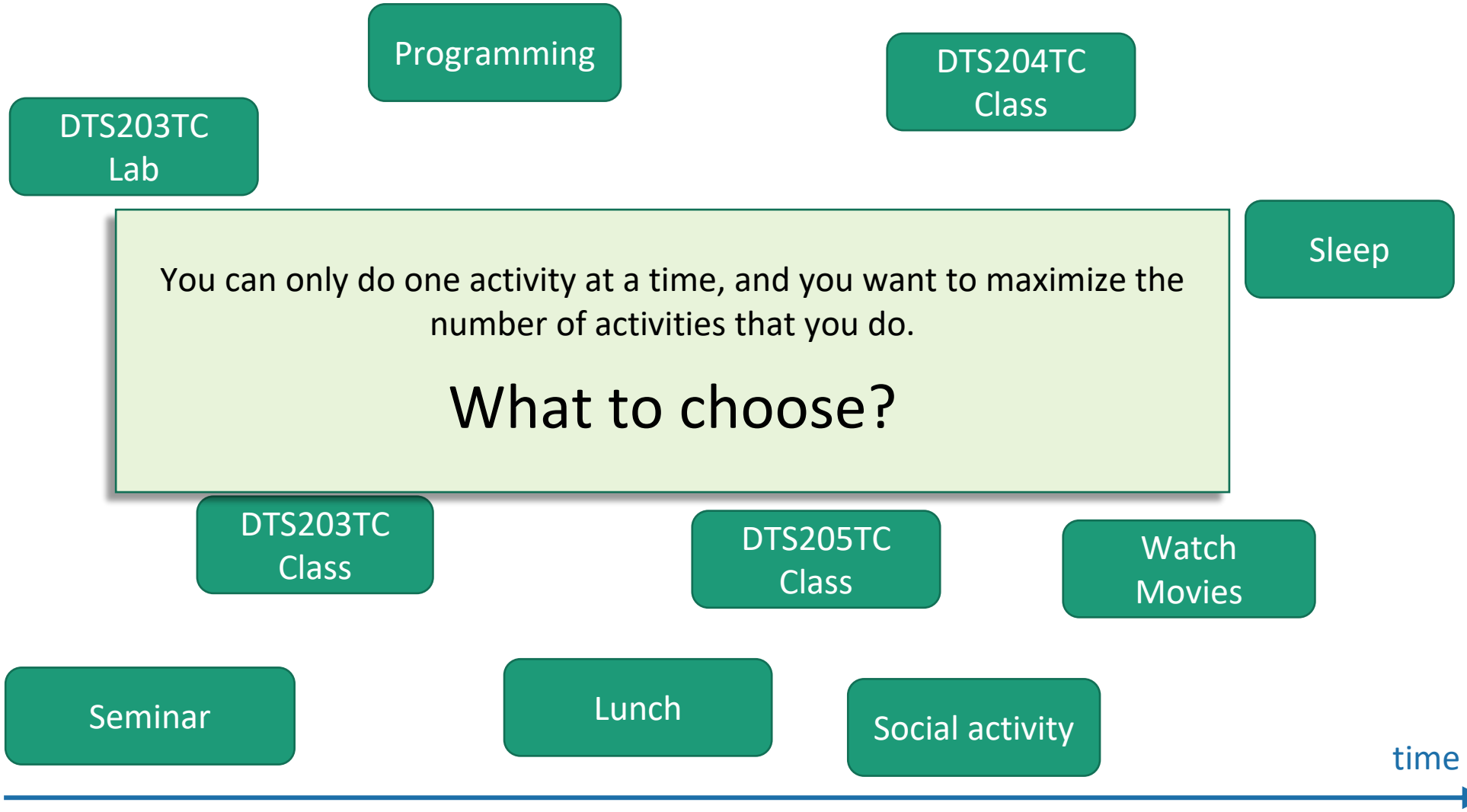
- For example:
  - For N = 11 and S = {1, 2, 5}, Best solutions: {1,5,5}. So output should be 3.
  - For N = 18 and S = {1, 2, 5}, Best solutions: {1,2,5,5,5}. So the output should be 5.

Coin Change

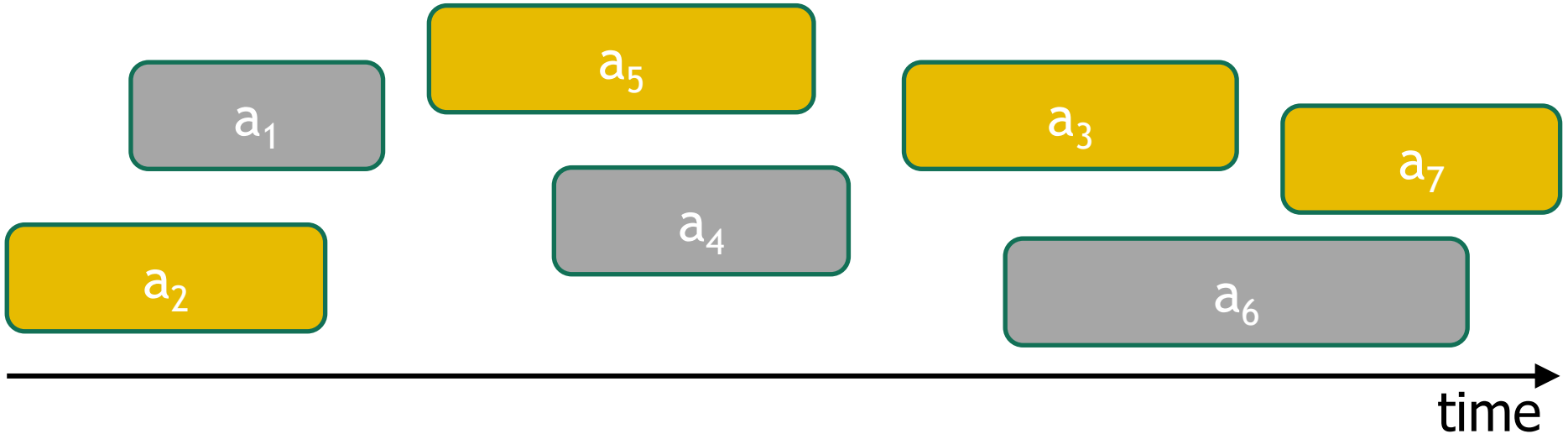
```python
def coin_change(n, s):
    s.sort(reverse=True)
    num_coins = 0
    i = 0
    while n > 0 and i < len(s):
        if s[i] <= n:
            num_coins += n // s[i]
            n %= s[i]
        i += 1
    if n == 0:
        return num_coins
    else:
        return -1

s = [1,2,5]
n = 18
print(coin_change(n,s))
```

```
5
```

Activity Selection



Activity Selection



- Pick activity you can add with the smallest finish time.
- Repeat.

Activity Selection

- Selecting a maximum set of non-overlapping activities from a given set of activities. Suppose there are n activities $A = [a_1, a_2, \ldots, a_i, \ldots, a_n]$. Each activity, e.g. $a_i = [start, end]$, has a start time and an end time, and the goal is to select as many activities as possible such that no two selected activities overlap in time.

- For example:
  - The output for activities = [[2, 5], [1, 3], [6, 9], [4, 8], [5, 7], [8, 10]] should be 3

Activity Selection

```python
def activity_selection(activities):
    activities = sorted(activities, key=lambda x: x[1])  # sort activities by their end times
    count = 0
    end_time = 0
    for start, end in activities:
        if start >= end_time:
            count += 1
            end_time = end
    return count

activities = [[2, 5], [1, 3], [6, 9], [4, 8], [5, 7], [8, 10]]

max_activities = activity_selection(activities)
print(max_activities)
```

```
3
```

Exercise 1: Fractional Knapsack

- Given: A set of n items, with each item i having
  - w_i – a positive weight
  - v_i – a positive benefit value

- Goal: Choose items with maximum total value but with weight at most W.

- You can take **fractions** of items (i.e., items are divisible)

Exercise 1: Fractional Knapsack Greedy Strategy

- Calculate value/weight ratio for each item
- Sort items by this ratio in descending order
- Pick items starting from the highest ratio:
  - If the item fits, take all
  - If not, take the fraction that fits

Exercise 2: Minimum Number of Platforms

Problem Statement

- You're given two lists:
  - arrival[]: arrival times of trains
  - departure[]: departure times of the same trains

Goal:

- Find the minimum number of platforms needed so that no train waits, i.e., no two trains are on the same platform at the same time.

Key Insight:

- When two trains overlap, they need different platforms.
- So, the problem becomes one of interval overlap counting.

https://www.geeksforgeeks.org/minimum-number-platforms-required-railwaybus-station/

Exercise 2: Minimum Number of Platforms

- You're given two lists:
  - arrival[] = [900, 940, 950, 1100, 1500, 1800],
  - departure[] = [910, 1200, 1120, 1130, 1900, 2000]

- Strategy

1. Sort the arrival[] and departure[] arrays.
2. Use two pointers:
   - i: tracking arrive
   - j: tracking depart
3. Move through time:
   - If arrival[i] <= departure[j]: → train arrives → need one more platform
   - Else: → train departs → free a platform

Time	Event Type	Total Platforms Needed at this Time
9:00	Arrival	1
9:10	Departure	0
9:40	Arrival	1
9:50	Arrival	2
11:00	Arrival	3
11:20	Departure	2
11:30	Departure	1
12:00	Departure	0
15:00	Arrival	1
18:00	Arrival	2
19:00	Departure	1
20:00	Departure	0